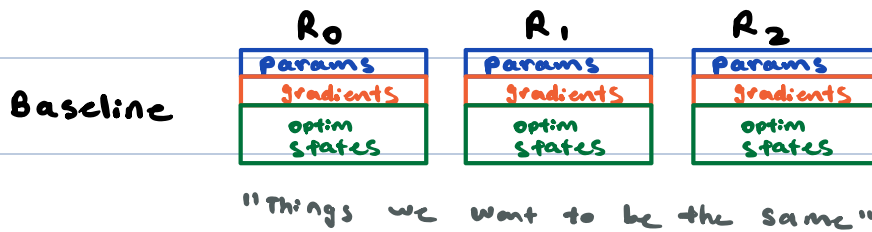


Data Parallelism & ZeRO

40-ddp



Rank 0

X-local
y-local

This is our sharded data:
different data on every rank!

Params = $\begin{bmatrix} W_1 \\ b_1 \\ W_2 \\ b_2 \end{bmatrix}$

flatten-dense-tensors

flat_params = $[W_1, b_1, W_2, b_2]$
1D contiguous array

broadcast

Rank 1

Rank 2

unflatten-dense-tensors

Params = $\begin{bmatrix} W_1 \\ b_1 \\ W_2 \\ b_2 \end{bmatrix}$

"Every rank has the same Params now"

Repeat:

flatten
broadcast
unflatten

from rank 0

for exp-avg
exp-avg-se

Broadcast
Parameters
to ranks

X-local y-local Params = $\begin{bmatrix} W_1 \\ b_1 \\ W_2 \\ b_2 \end{bmatrix}$ exp-avg exp-avg-se

out = Linear(ReLU(Linear(...)))

loss = mse_loss(out, y-local)
loss.backward()

grads = [p.grad for p in params]

flatten-dense-tensors

flat_grad

AllReduce gradients across ranks

unflatten

everyone now has updated p.grad
Perform optimizer step for every parameter!

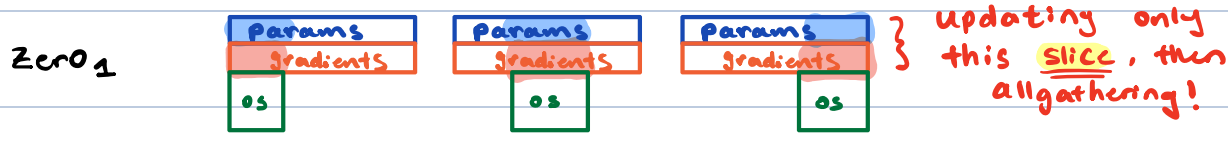
Run Fwd + Bwd
Pass

Allreduce gradients

41 - Zero 1

- same as DDP but shard optimizer state

FP32 master weights
FP32 Adam 1st moment
FP32 Adam 2nd moment



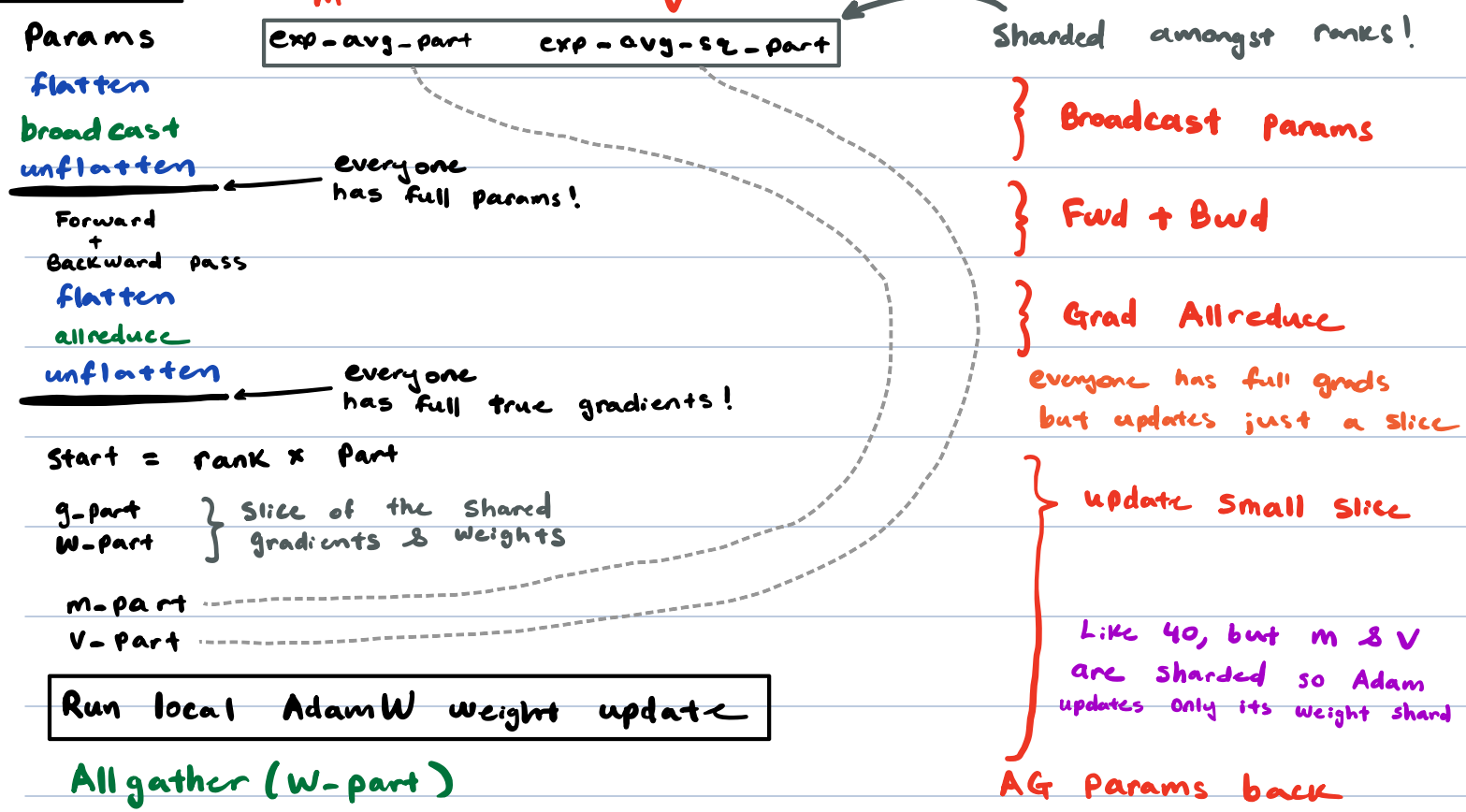
Review of Adam:

$$W_{t+1} = W_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda W_t \right)$$

$\hat{m}_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$
 $\hat{v}_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
 $\epsilon \leftarrow 10^{-8}$

Adam stores states $[W_t, m_t, v_t]$ for every param!

Rank X



now everyone has full updated weights!

- In real Zero 1, there's master FP32 weights that're also sharded!
 ↳ Reconstruct & free parameters / gradients as needed
 ↳ integrated into a training loop

- This is deepspeed's Zero 1: gradients are NOT sharded

42 - zero2 - optimizer



Rank X

Params

flatten

broadcast

unflatten

Forward

Backward pass

flatten

reduce-scat (g-part)

unflatten

Start = rank * Part

g-part

w-part

} Slice of the shared gradients & weights

m-part

v-part

Run local AdamW weight update

Allgather (w-part)

now everyone has full updated weights!

exp-avg-part exp-avg-sz-part g-part

Sharded amongst ranks!

} Broadcast params

} Fwd + Bwd

} Grad REDUCE SCATTER

everyone has a sharded slice of the gradients

} update small slice

AG Params back

- "REAL" Zero₂:

After doing bwd pass:

↳ Reduce & P2P send to right rank

↳ Free when not needed

Each rank updates their gradient slice shard

Allgather the parameters

=

TBD...

Coming soon...

ZERO₃ / FSDP Visuals

Here, we also shard params!

Zero₃



This uses:

- 2 allgathers
- 1 reduceScatter

Optimized via:

- freezing requested Params / gradients
- comp. comm. overlap

Ring Attention

Attention is 3 phases:

- Find row-max m
- Let $S = \sum e^{\text{row-elem} - m}$
- Divide each row-elem by S

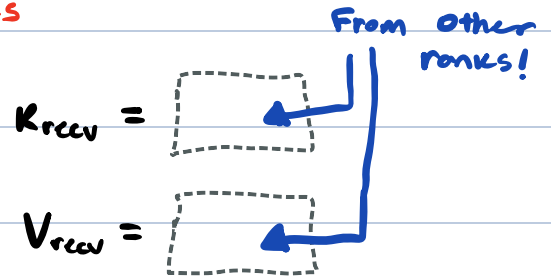
Each rank has:

$$B = \text{seqLen} / \# \text{ranks}$$

$$Q_{\text{local}} = d \times \boxed{}$$

$$K_{\text{local}} = d \times \boxed{}$$

$$V_{\text{local}} = d \times \boxed{}$$



$$\overrightarrow{\text{running-max}} = 1 \times \boxed{ B}$$

$$\overrightarrow{\text{running-denom-sum}} = 1 \times \boxed{ B}$$

$$\text{running out accum} = d \times \boxed{}$$

$$\text{Let } S_{\text{block}} = \frac{1}{\sqrt{d}} \times (Q_{\text{local}} \cdot K_{\text{recv}}^T) = [\dots]_{B \times B}$$

↓ rowmax

$$\overrightarrow{\text{local-max}} = [\vdots]_{B \times 1}$$

$$\begin{aligned} \overrightarrow{\text{new-max}} &= \max(\overrightarrow{\text{running-max}}, \overrightarrow{\text{local-max}}) \\ &= [\vdots]_{B \times 1} \end{aligned}$$

$$\text{Running Correction Factor} = e^{(\overrightarrow{\text{running-max}} - \overrightarrow{\text{new-max}})} = [\vdots]_{B \times 1}$$

$$\text{Local Correction Factor} = e^{(\overrightarrow{\text{local-max}} - \overrightarrow{\text{new-max}})} = [\vdots]_{B \times 1}$$

Let's get the exps for our new block:

$$\begin{bmatrix} \text{final} \\ \text{output} \\ \text{intermediate} \\ \text{exps} \end{bmatrix}_{B \times B} = e^{\left(\underset{B \times B}{S_{\text{block}}} - \underset{B \times 1}{\text{new_max}} \right)}$$

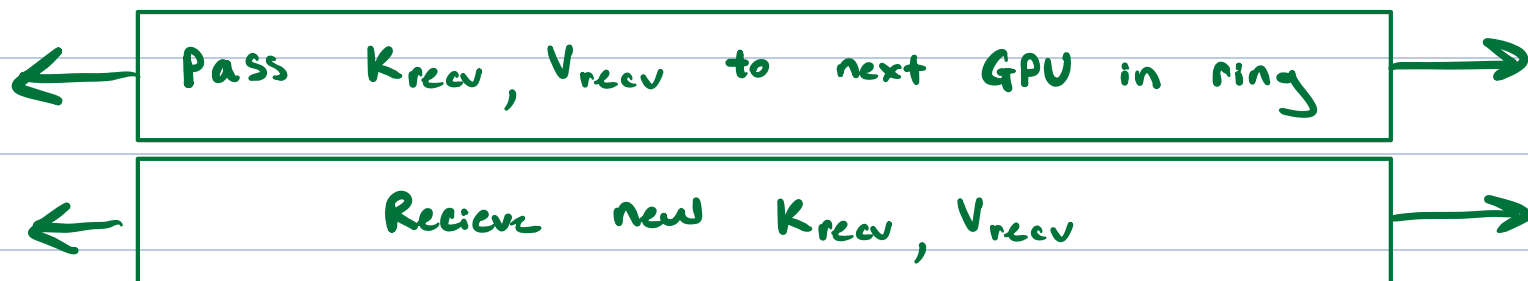
Subtract
rowwise max
& exponentiate
matrix elements

↑
Still need to
divide Σe !

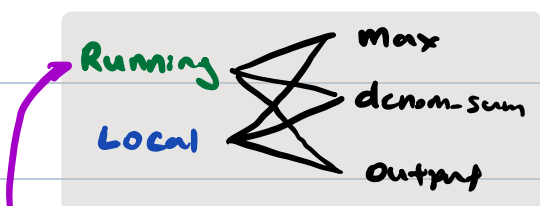
$$\underset{B \times 1}{\text{final_denom_sum}} = \left(\underset{B \times 1}{\text{Running Correction Factor}} \odot \underset{B \times 1}{\text{running_denom_sum}} \right) + \sum_{\text{rows}} \underset{B \times 1}{\begin{bmatrix} \text{final} \\ \text{output} \\ \text{intermediate} \\ \text{exps} \end{bmatrix}_{B \times B}}$$

$$\underset{B \times d}{\begin{bmatrix} \text{final_numerators} \end{bmatrix}} = \left(\underset{B \times 1}{\text{Running Correction Factor}} \odot \underset{B \times d}{\text{running_out_accum}} \right) + \left(\underset{B \times B}{\begin{bmatrix} \text{final} \\ \text{output} \\ \text{intermediate} \\ \text{exps} \end{bmatrix}} \cdot \underset{B \times d}{V_{\text{row}}} \right)$$

$$\underset{B \times d}{\text{running_out_accum}} = \underset{B \times d}{\begin{bmatrix} \text{final_numerators} \end{bmatrix}} / \underset{B \times 1}{\text{final_denom_sum}}$$



Key behind RA:



Use a correction factor for this!

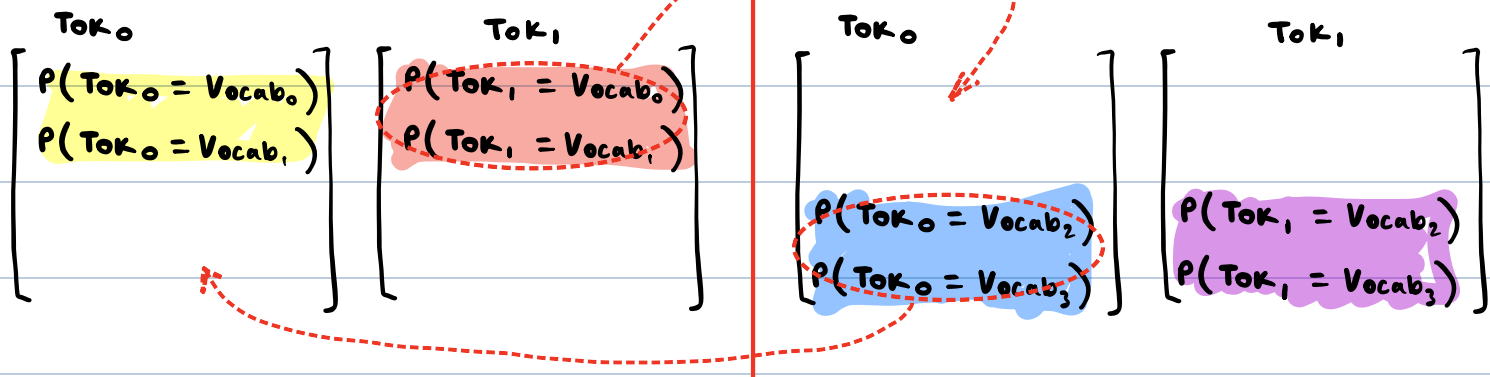
82 - Vocab - Parallel - top-K

At the very end, our model outputs:

"Vocabulary parallel" Format

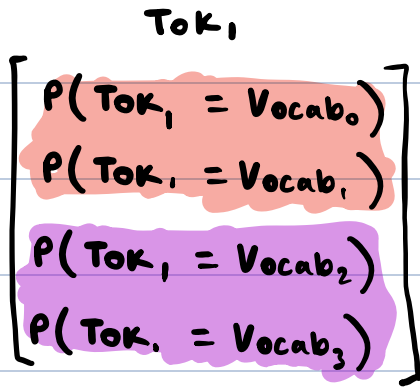
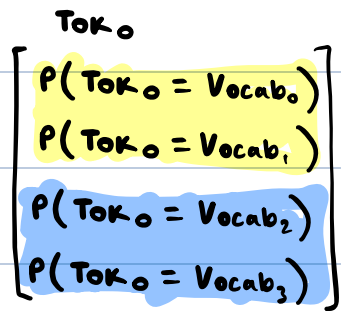
RANK 0

RANK 1



A2A - exchange

"Sequence parallel" Format



↓
Top K
softmax
gather (get target prob.)

↓
Top K
softmax
gather (get target prob.)

$P(\text{Tok}_0 = \text{Vocab}_3)$

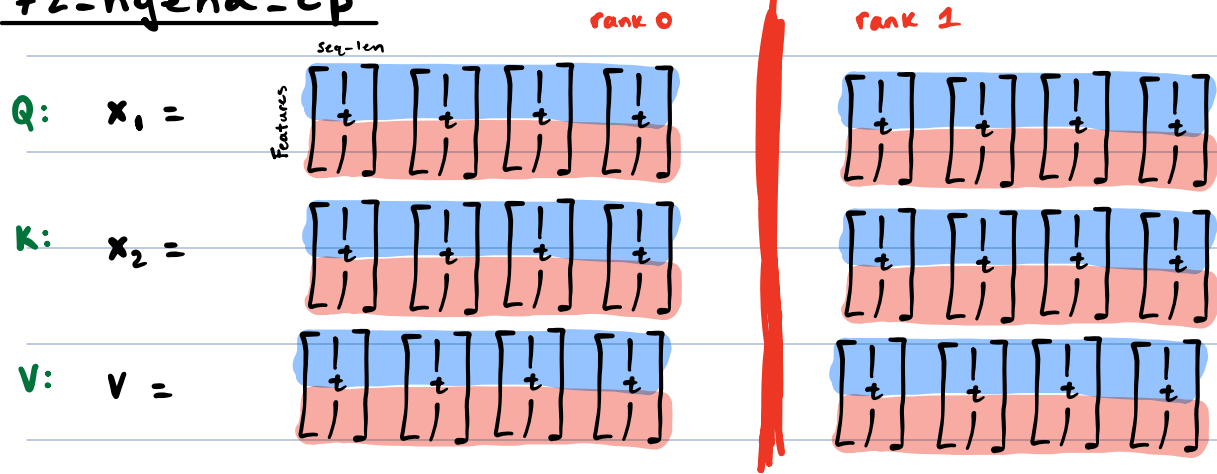
$P(\text{Tok}_1 = \text{Vocab}_2)$

allgather

$P(\text{Tok}_0 = \text{Vocab}_3)$ $P(\text{Tok}_1 = \text{Vocab}_2)$

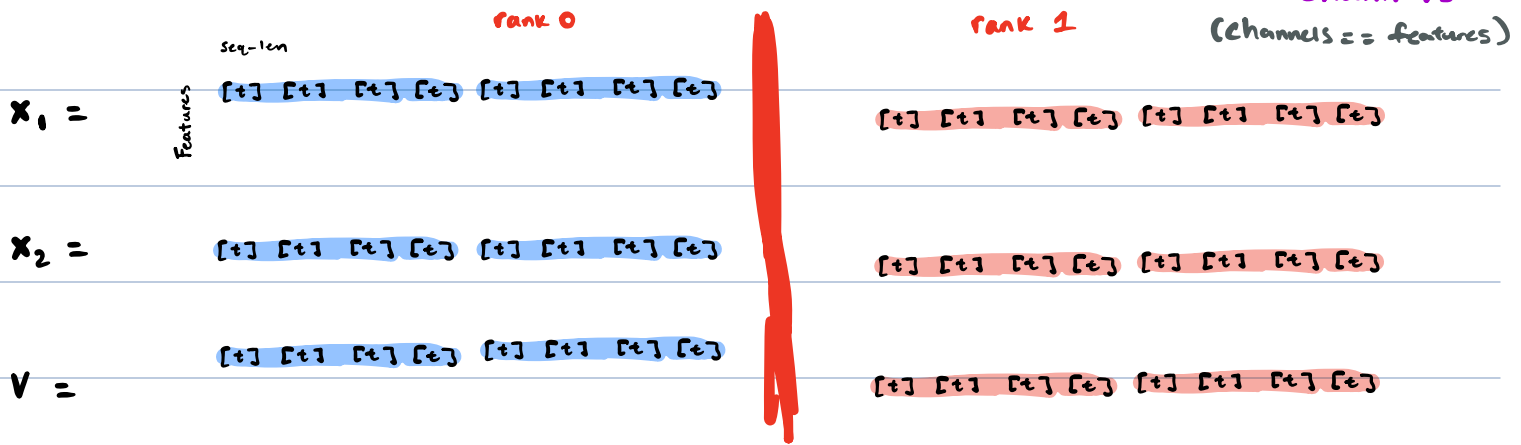
$P(\text{Tok}_0 = \text{Vocab}_3)$ $P(\text{Tok}_1 = \text{Vocab}_2)$

72-hyena-cp



A2A - SPLIT - FULL

full
seq. diff
channels



$$z = x_2 \odot v$$

$$\text{FFT}(z)$$

$$z = x_1 \odot z$$

$$z = x_2 \odot v$$

$$\text{FFT}(z)$$

$$z = x_1 \odot z$$

$z\text{-seq} =$

A2A - FULL - SPLIT

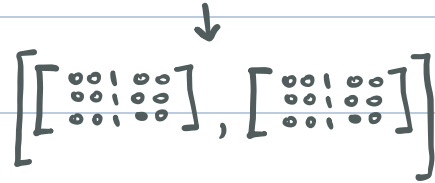
$z\text{-seq} =$

LLM optimizations

- Bypass A2A w/ NVLink

81-sam3_IoU-suppression

- each gpu has its own mask + confidence score



↓

$$[0.8, 0.9]$$

- Collect masks & scores

- ID overlapping boxes & mark those above IoU threshold

- Suppress (delete) occluded boxes ($>$ IoU threshold)



delete
the
occluded
one!

Rank₀

masks_{local} = $[[8], [\text{cloud}]]$

scores_L = $[0.5, 0.7]$

Rank₁

masks_{local} = $[[\text{red}]]$

scores_L = $[0.1]$

Variable - all-gather

last-occluded

masks_{global} = $[[8], [\text{cloud}], [\text{red}]]$

scores_{global} = $[0.5, 0.7, 0.1]$



Compute overlaps & IoU

- LLM bitpacks pixels into an int-32